

MLA0303_RL_PROGRAM_12.py

```
"""
Experiment 12: Policy-based Reinforcement Learning (REINFORCE with a
softmax policy) for an industrial robotic arm performing pick-and-place.
State = arm position along a 1-D rail (0..5); Actions = LEFT/RIGHT/PICK/PLACE
"""

import numpy as np

np.random.seed(0)

N_POS = 6
PICK_POS = 1
PLACE_POS = 4
ACTIONS = ["LEFT", "RIGHT", "PICK", "PLACE"]
GAMMA = 0.95
LR = 0.05
EPISODES = 3000

class ArmEnv:
    def reset(self):
        self.pos = 0
        self.holding = False
        return self._state()

    def _state(self):
        return self.pos * 2 + int(self.holding)

    def step(self, action):
        reward, done = -1, False
        if action == 0: # LEFT
            self.pos = max(0, self.pos - 1)
        elif action == 1: # RIGHT
            self.pos = min(N_POS - 1, self.pos + 1)
        elif action == 2: # PICK
            if self.pos == PICK_POS and not self.holding:
                self.holding = True
                reward = 5
            else:
                reward = -3
        elif action == 3: # PLACE
            if self.pos == PLACE_POS and self.holding:
                self.holding = False
                reward = 20
                done = True
            else:
                reward = -3
        return self._state(), reward, done

N_STATES = N_POS * 2
theta = np.zeros((N_STATES, len(ACTIONS))) # policy parameters

def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

def choose_action(state):
    probs = softmax(theta[state])
    return np.random.choice(len(ACTIONS), p=probs), probs

def train():
    env = ArmEnv()
    reward_history = []
    baseline = 0.0
    for ep in range(EPISODES):
        state = env.reset()
        trajectory = []
        for _ in range(40):
            action, probs = choose_action(state)
            next_state, reward, done = env.step(action)
            trajectory.append((state, action, reward, probs))
            state = next_state
            if done:
                break
        # Compute discounted returns
        G = 0
        returns = [0] * len(trajectory)
        for t in reversed(range(len(trajectory))):
            G = trajectory[t][2] + GAMMA * G
            returns[t] = G
        episode_return = sum(r for _, _, r, _ in trajectory)
        baseline = 0.95 * baseline + 0.05 * episode_return # running-average baseline
        # REINFORCE policy-gradient update (advantage = return - baseline)
        for (s, a, r, probs), G_t in zip(trajectory, returns):
            advantage = G_t - baseline
            grad = -probs
            grad[a] += 1
            theta[s] += LR * advantage * grad
        reward_history.append(episode_return)
    return env, reward_history

if __name__ == "__main__":
    env, rewards = train()
    print(f"Average return (first 50 episodes): {np.mean(rewards[:50]):.2f}")
```

```
print(f"Average return (last 50 episodes) : {np.mean(rewards[-50:]).2f}")

state = env.reset()
path = [(env.pos, env.holding)]
for _ in range(20):
    action = int(np.argmax(theta[state]))
    state, _, done = env.step(action)
    path.append((env.pos, env.holding))
    if done:
        break
print("Learned pick-and-place sequence (pos, holding):", path)
```

PROGRAM OUTPUT

Average return (first 50 episodes): -4.32

Average return (last 50 episodes) : 20.94

Learned pick-and-place sequence (pos, holding): [(0, False), (1, False), (1, True), (2, True), (3, True), (4, True),